Web Engineering & Development (SWE 363)

Advanced React

In today's lecture:

- Styling
- React Bootstrap
- Fetching Data

Reference:

- Zybook: 5.15-5.17

5.15 Styling (React)

Three Ways to Style React Components

Key Concepts:

- **Inline**: `style={{ ... }}` — camelCase CSS props (e.g. `backgroundColor`)
- **CSS file**: `import "./App.css"` + `className="..."` (not `class`)
- **CSS Modules**: `import styles from './X.module.css'` → scoped class names

```
<div style={{ backgroundColor: 'blue', padding: '20px' }}>Inline</div>

import "./Welcome.css";
<div className="my-card">CSS file</div>

import styles from './Card.module.css';
<div className={styles.card}>CSS Module</div>
```

CSS Variables & Responsive Design

```css
:root {
  --primary-color: #0d6efd;
  --light-color: #f8f9fa;
}
.my-button { background-color: var(--primary-color); }

/* Mobile first, then larger screens */
.user-avatar { width: 50px; }
@media (min-width: 768px)  { .user-avatar { width: 60px; } }
@media (min-width: 1024px) { .user-avatar { width: 80px; } }
```

- **CSS variables**: define once in `:root`, reuse with `var(--name)`
- **Media queries**: change styles by viewport width

5.16 React Bootstrap

What is React Bootstrap?

Bootstrap = CSS framework (pre-built, responsive components)

React-Bootstrap = Bootstrap as React components (props instead of only classes)

```
npm install react-bootstrap bootstrap
```

```
<!-- public/index.html -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
```

React Bootstrap docs

Layout & Utility Classes

```
import { Container, Row, Col } from 'react-bootstrap';

<Container>
  <Row>
    <Col md={6} lg={4}>Column 1</Col>
    <Col md={6} lg={4}>Column 2</Col>
    <Col md={12} lg={4}>Column 3</Col>
  </Row>
</Container>
```

Utilities (className):

- Spacing: `mb-4`, `mt-5`, `py-3`, `px-2`
- Color: `bg-primary`, `text-white`, `bg-light`
- Layout: `d-flex`, `justify-content-center`, `align-items-center`, `text-center`

5.17 Fetching Data

The Problem: When to Fetch?

You already know `fetch()` — but **when** do you call it in a component?

```
function Users() {
  // ❌ Don't fetch here — runs on every render, can cause infinite loops
  fetch('https://jsonplaceholder.typicode.com/users')
    .then(r => r.json())
    .then(data => setUsers(data));

  return <div>...</div>;
}
```

Need: run the fetch **once** after the component appears on the screen → use `useEffect`

What is `useEffect`?

`useEffect` = run **side effects** after React paints the UI (fetching, timers, subscriptions)

```
import { useEffect } from 'react';
useEffect(() => {
  // code here runs AFTER the first render
  console.log('Component mounted');
}, []);
```

- **1st argument**: a function — the effect to run
- **2nd argument**: **dependency array** — when to re-run
  - `[]` → run **once** after mount
  - `[userId]` → re-run when `userId` changes
  - omitted → run after **every** render (usually avoid)

State for Fetched Data

Use **three pieces of state** while waiting for the API:

```
import { useState } from 'react';

const [users, setUsers] = useState([]);       // the data
const [loading, setLoading] = useState(true); // still waiting?
const [error, setError] = useState(null);     // something went wrong?
```

State	Purpose
`users`	Store the JSON you get back
`loading`	Show a spinner / “Loading…”
`error`	Show an error message

Putting It Together

```
import { useState, useEffect } from 'react';

useEffect(() => {
  const fetchUsers = async () => {
    try {
      setLoading(true);
      const response = await fetch('https://jsonplaceholder.typicode.com/users');
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      setUsers(await response.json());
    } catch (err) { setError(err.message);
    } finally { setLoading(false);
    }
  };
  fetchUsers();              // call the async function from inside the effect
}, []);                      // [] = fetch once when the component mounts
```

Note: `useEffect` itself cannot be `async` — define an `async` function **inside**, then call it.

Loading, Error & Empty UI

Always handle all three states when rendering fetched data:

```
{loading && <p>Loading users...</p>}

{error && <Alert variant="danger">{error}</Alert>}

{!loading && !error && users.length === 0 && (
  <Alert variant="info">No users found.</Alert>
)}

{!loading && !error && users.map(user => (
  <div key={user.id}>{user.name}</div>
))}
```

30m

Demo

5.6 React Styling & Fetching